

Nghiên cứu và Thiết kế Hệ thống Thị giác Máy tính Nhúng On-Device trên ESP32-CAM cho Trạm Quan trắc Hạt và Rác trong Nguồn Nước

Kiến trúc mô hình và các lựa chọn tối ưu trên hệ thống nhúng ESP32-CAM

Việc triển khai các mô hình thị giác máy tính trên bo mạch ESP32-CAM sử dụng vi xử lý Xtensa LX6 hai nhân chạy ở tần số 240MHz đòi hỏi một chiến lược tối ưu hóa kiến trúc cực kỳ nghiêm ngặt. LX6 là kiến trúc không hỗ trợ các tập lệnh SIMD (Single Instruction Multiple Data), không có bộ tăng tốc mạng thần kinh phần cứng (NN Accelerator), và hoàn toàn không được hưởng lợi từ thư viện tăng tốc vector ESP-NN vốn chỉ dành riêng cho nhân Xtensa LX7 của dòng ESP32-S3. Mọi phép tính tích chập lượng tử hóa số nguyên 8-bit (INT8 Conv) trên LX6 đều phải thực thi thông qua các phép toán số học vô hướng cơ bản của đơn vị logic đại số (ALU). Do đó, việc lựa chọn cấu trúc mô hình quyết định tính khả thi của toàn bộ hệ thống quan trắc.

1. Khả năng khả thi của các bộ phát hiện đối tượng end-to-end nhỏ (YOLO-nano, PicoDet, NanoDet, TinySSD)

Các mô hình phát hiện đối tượng tích hợp một giai đoạn (one-stage detectors) được lượng tử hóa hoàn toàn mang lại sự tiện lợi lớn nhờ khả năng xuất trực tiếp bounding box và phân lớp đối tượng trong một lần suy luận. Tuy nhiên, khi đánh giá định lượng trên nhân Xtensa LX6 của ESP32-CAM, phương án này bộc lộ những hạn chế chí mạng về cả dung lượng bộ nhớ lẫn tốc độ thực thi.

Dưới đây là bảng ước tính định lượng hiệu năng của một mô hình YOLOv8-nano tối thiểu hóa (độ phân giải đầu vào 160 $\times$ 160 pixel, Grayscale, cấu hình 1 lớp đối tượng) khi chạy trên lõi vô hướng Xtensa LX6:

Chỉ số hiệu năng	Giá trị ước tính định lượng	Nguồn tham chiếu / Cơ sở tính toán
Độ phức tạp tính toán	~0.18 GMACs (Giga Multiply-Accumulate)	Ước tính từ kiến trúc YOLOv8-nano thu gọn
Kích thước mô hình (INT8)	1.8 MB đến 3.2 MB	Dựa trên các mô hình nén hệ COCO
Kích thước Tensor Arena	650 KB đến 900 KB	Đo đạc thực tế từ đồ thị tính toán TFLite Micro
Độ trễ suy luận (SRAM nội)	Không thể thực hiện (Không đủ dung lượng SRAM)	Heap SRAM trống thực tế chỉ còn 150-250KB
Độ trễ suy luận (PSRAM ngoài)	12.5 đến 18.2 giây	Ước tính từ tốc độ bus QSPI 80MHz của PSRAM ngoài

Do bộ đệm kích hoạt (Tensor Arena) vượt quá dung lượng SRAM nội tự do (chỉ còn khoảng

150-250KB sau khi trừ đi driver camera và WiFi), hệ thống bắt buộc phải cấp phát Tensor Arena trên PSRAM ngoài thông qua hàm ps_malloc(). Băng thông truyền tải của PSRAM QSPI hoạt động ở tần số 80MHz cực kỳ giới hạn (tối đa chỉ khoảng 40MB/s). Việc CPU liên tục truy xuất các activation tensor trung gian trên PSRAM ngoài với độ trễ cao (10-20 chu kỳ CPU cho mỗi lần truy cập so với 1-2 chu kỳ trên SRAM nội) tạo ra một điểm nghẽn vật lý nghiêm trọng. Hệ quả là thời gian suy luận bị kéo dài lên tới hơn 12 giây cho một khung hình đơn lẻ. Mặc dù chu kỳ vận hành Stop-Flow cho phép thời gian xử lý kéo dài vài giây giữa các mẻ đo, việc CPU chạy ở công suất tối đa trong hơn 12 giây liên tục sẽ đẩy nhiệt độ chip lên rất cao, gây sụt áp nguồn (brownout) và triệt tiêu hoàn toàn khả năng xử lý song song các tác vụ điều khiển thiết bị ngoại vi khác.

2. Pipeline lai kết hợp xử lý ảnh cổ điển và phân loại cục bộ (Hybrid Pipeline)

Để khắc phục triệt để giới hạn vật lý của ESP32-CAM, giải pháp khả thi nhất là cấu trúc một pipeline lai bao gồm hai giai đoạn tách biệt:

- Giai đoạn 1: Phát hiện và đo kích thước bằng thị giác máy tính cổ điển (Classical CV):** Nhờ thiết lập chiếu sáng ngược (backlit silhouette), hình ảnh thu được có độ tương phản cực kỳ cao. Các hạt xuất hiện dưới dạng bóng đen sắc nét trên nền sáng đều. Hệ thống thực hiện chuyển đổi ảnh chụp từ camera sang thang xám (Grayscale VGA 640 $\times$ 480), áp dụng thuật toán nhị phân hóa Otsu thích nghi để tự động tìm ngưỡng tối ưu. Sau đó, thuật toán phân tích thành phần liên thông (Connected Components Labeling - CCL) kết hợp cấu trúc dữ liệu Union-Find được thực thi trực tiếp trên ảnh nhị phân. Thuật toán này trích xuất chính xác số lượng hạt, tọa độ bounding box, diện tích bề mặt (tổng số pixel của blob) và chu vi của từng hạt. Toàn bộ giai đoạn này hoạt động tuần tự theo dòng quét ảnh (line-by-line), chỉ yêu cầu bộ đệm dòng cực nhỏ (<5KB) trong SRAM nội và hoàn thành trong thời gian từ **120ms đến 250ms** đối với ảnh VGA.
- Giai đoạn 2: Phân loại hình thái bằng mô hình siêu nhỏ (Tiny Classifier):** Từ danh sách các bounding box trích xuất ở Giai đoạn 1, hệ thống thực hiện cắt các vùng ảnh tương ứng (crop). Vùng ảnh crop của từng hạt được chuẩn hóa về kích thước cố định (ví dụ: 32 $\times$ 32 pixel) và đưa vào một mạng tích chập siêu nhỏ (Tiny CNN) hoặc một mạng perceptron đa tầng (MLP) hoạt động trên các đặc trưng hình học trích xuất thủ công.

Bảng so sánh chi tiết giữa bộ phát hiện đối tượng end-to-end (YOLOv8-nano) và Pipeline lai (A2) thể hiện ưu thế tuyệt đối của giải pháp lai:

Chỉ số so sánh	YOLOv8-nano (Kiến trúc A1)	Pipeline lai (Kiến trúc A2)
Độ trễ toàn hệ thống	12.0 - 18.0 giây	250 - 550 ms (đối với mẻ có 15 hạt)
Dung lượng Tensor Arena	650 KB - 900 KB (bắt buộc nằm trong PSRAM)	12 KB - 24 KB (nằm hoàn toàn trong SRAM nội)
Dung lượng Flash tĩnh	~2.5 MB	25 KB - 60 KB
Độ chính xác đo kích thước	Kém (sai số lớn do imgsz bị nén mạnh)	Tuyệt đối (đo trực tiếp trên ảnh gốc VGA)
Độ ổn định hệ thống	Thấp (nhiệt độ tăng cao, dễ sụt áp nguồn)	Cực kỳ cao (CPU hoạt động theo chu kỳ ngắn)

3. Phân tích giải pháp FOMO (Faster Objects, More Objects) và kỹ

thuật lấy lại kích thước

Mô hình FOMO của Edge Impulse được thiết kế chuyên biệt cho việc phát hiện vật thể trên các dòng vi điều khiển tài nguyên thấp bằng cách chuyển đổi bài toán phát hiện thành bài toán phân loại theo cấu trúc lưới (grid-based classification). Một mô hình FOMO sử dụng backbone MobileNetV2 (96 $\times$ 96 Grayscale, alpha 0.35) thực thi trên ESP32 đạt độ trễ khoảng **862ms**.

Tuy nhiên, FOMO loại bỏ các lớp hồi quy bounding box để tiết kiệm tài nguyên, dẫn đến việc hệ thống chỉ nhận được tọa độ tâm (centroid) của ô lưới chứa vật thể mà hoàn toàn mất đi thông tin về kích thước và hình dáng thực tế của hạt. Điều này vi phạm trực tiếp yêu cầu đo đạc phân bố kích thước hạt (size distribution) của ứng dụng.

Sự đánh đổi này chỉ có thể chấp nhận được khi bài toán chuyển sang một kịch bản đơn giản hơn, nơi kích thước của các hạt trong từng phân lớp là hoàn toàn cố định hoặc không cần khảo sát sâu. Để lấy lại kích thước hạt gần đúng từ đầu ra centroid của FOMO, có hai phương án kỹ thuật khả thi:

- **Phương án 1: Hiệu chỉnh thống kê theo lớp (Class-based Statistical Mapping):** Nếu các lớp đối tượng có kích thước vật lý đặc trưng không đổi (ví dụ: bọt khí luôn có đường kính trung bình 1.2mm), hệ thống có thể gán trực tiếp kích thước giả định dựa trên kết quả phân loại của FOMO. Phương án này có sai số rất lớn đối với các dị vật bất định hình như chất hữu cơ hay hạt nhựa vỡ.
- **Phương án 2: Thuật toán loang vùng cục bộ (Localized Region Growing / Flood Fill):** Sử dụng tọa độ centroid do FOMO cung cấp làm điểm mồi (seed point), hệ thống ánh xạ ngược lại tọa độ này lên ảnh VGA gốc độ phân giải cao. Tại vùng lân cận điểm mồi, thực hiện thuật toán loang vùng nhị phân cục bộ (Local Flood Fill) để xác định biên giới của hạt. Kỹ thuật này giúp khôi phục chính xác bounding box và diện tích blob mà không cần quét lại toàn bộ ảnh gốc, nhưng làm tăng độ phức tạp của thuật toán điều khiển.

4. Khả năng thay thế CNN bằng các đặc trưng hình học thủ công kết hợp bộ phân loại cực nhỏ

Ảnh thu được từ hệ thống chiếu sáng ngược (backlit silhouette) cung cấp cấu trúc hình học cực kỳ rõ nét. Các đặc trưng hình thái phi tham số có thể dễ dàng tính toán từ ma trận Connected Components:

- **Diện tích (A):** Tổng số pixel thuộc về blob.
- **Chu vi (P):** Độ dài đường biên của hạt.
- **Độ tròn (Circularity - C):** $C = \frac{4 \times \text{start_span} \times \text{end_span}}{\pi A^2}$
- **Tỷ lệ khung hình (Aspect Ratio - AR[span_73](start_span)[span_73](end_span)):** $AR = \frac{W_{\text{bbox}}}{H_{\text{bbox}}}$
- **Độ đặc (Solidity - S):** Tỷ lệ giữa diện tích blob và diện tích bao lồi (Convex Hull Area): $S = \frac{A}{A_{\text{convex}}}$
- **Tỷ lệ cường độ sáng trung tâm (Center-to-Edge Intensity Ratio - $R_{\{I\}}$):** Bọt khí (bubble) khi chịu tác động của ánh sáng ngược sẽ hoạt động như một thấu kính hội tụ, tạo ra một điểm sáng mạnh ở chính tâm của silhouette tối. Chỉ số $R_{\{I\}}$ đo tỷ lệ giữa cường độ sáng của 4 pixel trung tâm centroid và cường độ sáng trung bình của đường biên blob.

Một bộ phân loại phi tuyến tính cực nhỏ như cây quyết định (Decision Tree) hoặc mạng MLP cấu hình $5 \times 8 \times 4$ hoạt động trên các đặc trưng trên chỉ chiếm chưa đầy 1.5KB ROM và thực thi dưới 0.5ms. Phương án này hoàn toàn đủ năng lực để phân loại chính xác các lớp đối tượng có cấu trúc hình học đặc trưng biệt lập: bọt khí (circularity $C \approx 1.0$, $R_{\{l\}} \gg 1.0$) và sợi fiber (aspect ratio $AR \geq 4.5$).

Tuy nhiên, mô hình học sâu CNN là **bắt buộc** khi hệ thống cần phân biệt giữa **hạt nhựa vi thể (plastic)** và **mảnh vụn hữu cơ (organic)**. Cả hai lớp này đều có hình dạng bất định hình, circularity biến thiên rộng và aspect ratio tương đồng. Sự khác biệt giữa chúng nằm ở các chi tiết kết cấu bề mặt, độ trong suốt một phần (mức độ xám của silhouette) và các góc cạnh vi mô – những đặc trưng phi tuyến tính phức tạp mà các công thức hình học thủ công không thể biểu diễn một cách bao quát được.

Kết luận kiến trúc tối ưu (Sweet Spot)

Kiến trúc lai (**Hybrid Pipeline**) là "sweet spot" duy nhất đáp ứng hoàn hảo cả ba yêu cầu ĐẾM, đo KÍCH THƯỚC chính xác và PHÂN LOẠI hình thái trên ESP32-CAM:

- Độ chính xác đo kích thước tuyệt đối:** Khâu đo kích thước thực hiện trực tiếp trên ảnh VGA gốc bằng thuật toán Connected Components, đạt độ phân giải thực tế lên tới 14 px/mm (sai số phép đo $< 0.07\text{mm}$).
- Khả thi về mặt tài nguyên:** Tensor Arena cho bộ phân loại Tiny CNN (đầu vào ảnh crop 32×32 Grayscale) chỉ cần 16KB, hoàn toàn nằm trong SRAM nội tốc độ cao.
- Tối ưu hóa năng lượng:** Triệt tiêu hoàn toàn hiện tượng nghẽn bus QSPI của PSRAM, giúp CPU hoàn thành tác vụ cực nhanh và có thể chuyển sang trạng thái năng lượng thấp giữa các mẻ đo.

Chiến lược lựa chọn độ phân giải đầu vào

Độ phân giải hình ảnh quyết định giới hạn đo lường của hạt nhỏ nhất, nhưng lại ảnh hưởng theo hàm lũy thừa bậc hai tới nhu cầu dung lượng bộ nhớ và thời gian tính toán của CPU.

1. Đánh đổi giữa độ phân giải (Image Size) và khả năng phân giải hạt mịn

Hệ thống quan trắc yêu cầu đo các hạt có kích thước dưới 2mm, hướng tới thiết kế tối thiểu đạt 0.5mm. Tại khoảng cách làm việc cố định 40mm với ống kính nhà máy (factory lens), mật độ pixel thu được ở độ phân giải VGA (640×480) là khoảng 14 px/mm. Phân tích chi tiết mức độ đáp ứng vật lý của các độ phân giải được thể hiện trong bảng sau:

Độ phân giải chụp	Kích thước hạt 2.0 mm (pixel)	Kích thước hạt 0.5 mm (pixel)	Yêu cầu bộ nhớ Grayscale (1 byte/px)	Khả năng thực thi thuật toán Connected Components
VGA (640 $\times$ 480)	28 pixel (Đầy đủ chi tiết)	7.0 pixel (Đạt yêu cầu nhận diện biên)	307.2 KB	Khả thi cao (Sử dụng stream buffer hoặc PSRAM)
QVGA (320 $\times$ 240)	14 pixel (Mất chi tiết biên)	3.5 pixel (Dễ bị nhầm với nhiễu đốm)	76.8 KB	Dễ dàng (Hoạt động hoàn toàn trên SRAM nội)

Độ phân giải chụp	Kích thước hạt 2.0 mm (pixel)	Kích thước hạt 0.5 mm (pixel)	Yêu cầu bộ nhớ Grayscale (1 byte/px)	Khả năng thực thi thuật toán Connected Components
QQVGA (160 \times 120)	7 pixel (Biến dạng hình thái)	1.7 pixel (Hạt biến mất hoàn toàn)	19.2 KB	Cực kỳ dễ dàng (Nhưng không đáp ứng yêu cầu đo)

Nếu giảm độ phân giải đầu vào xuống mức QVGA hoặc QQQVGA để giảm tải cho các mạng thần kinh chuẩn, các hạt có đường kính nhỏ dưới 1.0mm sẽ bị tiêu biến hoặc biến dạng nghiêm trọng do lỗi răng cưa hóa, khiến phép đo phân bố kích thước hoàn toàn sai lệch. Do đó, việc duy trì độ phân giải ảnh chụp ở mức tối thiểu là **VGA** là yêu cầu bắt buộc đối với khâu đo kích thước.

2. Phân tích kỹ thuật chia ô ảnh (Tiling) để bảo toàn độ phân giải

Tiling là kỹ thuật chia nhỏ một bức ảnh lớn (ví dụ: ảnh SXGA 1280 \times 1024) thành các ô vuông nhỏ có kích thước tương thích với mạng CNN (ví dụ: 160 \times 160 pixel) với một khoảng chồng lấp cố định (overlap) để tiến hành suy luận tuần tự.

- **Đánh giá chi phí Latency:** Đối với một ảnh SXGA (1280 \times 1024), nếu chia thành các ô 160 \times 160 với độ chồng lấp 20 pixel ở các biên, hệ thống sẽ cần xử lý tổng cộng khoảng 70 ô. Ngay cả khi sử dụng một mô hình CNN siêu nhanh chạy mất 150ms trên một ô, tổng thời gian suy luận sẽ là: $T_{\text{tiling}} = 70 \times 150 \text{ ms} = 10.5 \text{ giây}$. Đây là chi phí độ trễ cực kỳ lớn, gây quá nhiệt hệ thống và tiêu tốn năng lượng nghiêm trọng.
- **Độ phức tạp của thuật toán ghép biên:** Khi một hạt nằm đè lên đường phân chia ô, nó sẽ bị cắt thành hai phần nằm ở hai ô khác nhau. Để ghép lại, hệ thống phải thực thi thuật toán tìm kiếm lân cận và tính toán chỉ số giao đè (IoU) trên hệ tọa độ toàn cục. Quá trình này đòi hỏi thuật toán phức tạp, dễ gây tràn bộ đệm khi mật độ hạt lớn.

3. Đề xuất chiến lược chụp và xử lý độ phân giải tối ưu

Giải pháp tối ưu nhất là **chụp ảnh ở độ phân giải VGA (640 \times 480) Grayscale**. Hệ thống sẽ không thực hiện kỹ thuật tiling cho khâu phát hiện. Thay vào đó, thuật toán Connected Components vô hướng sẽ quét tuần tự dòng ảnh VGA Grayscale trực tiếp từ PSRAM. Quá trình quét dòng này hoàn toàn không chiếm dụng dung lượng SRAM nội lớn vì nó xử lý theo cơ chế stream.

Sau khi xác định được các bounding box của hạt trên ảnh VGA gốc, hệ thống mới thực hiện cắt ảnh cục bộ (crop) trực tiếp từ bộ đệm VGA trong PSRAM và nội suy nhanh về kích thước 32 \times 32 Grayscale để đưa vào bộ phân loại Tiny CNN. Kỹ thuật này giúp bảo toàn nguyên vẹn độ phân giải vật lý của hạt nhỏ nhất mà không phải trả giá bằng chi phí thời gian suy luận khổng lồ của phương pháp tiling.

Chiến lược nén và lượng tử hóa mô hình

Để đảm bảo mô hình phân loại cục bộ chạy mượt mà trên SRAM nội của ESP32-CAM, việc tối

ưu hóa dung lượng thông qua nén và lượng tử hóa là bắt buộc.

1. Lượng tử hóa sau đào tạo (Post-Training Quantization - PTQ) Full-Integer INT8

PTQ chuyển đổi toàn bộ trọng số (weights) và các hàm kích hoạt (activations) từ định dạng số thực dấu phẩy động 32-bit (FP32) sang định dạng số nguyên 8-bit (INT8).

[Mô hình FP32] —► [Representative Dataset (200-500 mẫu)] —► [TFLite Converter] —► [Mô hình INT8]

- **Tập dữ liệu đại diện (Representative Dataset):** Để lượng tử hóa chính xác dải động của các hàm kích hoạt mà không làm sụp đổ độ chính xác của mô hình, hệ thống cần một tập dữ liệu hiệu chuẩn gồm **200 đến 500 ảnh crop thực tế** của các hạt. Tập dữ liệu này phải phản ánh chính xác phân bố cường độ sáng của hệ thống chiếu sáng ngược thực tế, bao gồm cả các biến thể nhiễu nền.
- **Mức sụt giảm độ chính xác thực tế:** Đối với mô hình phân loại ảnh hạt silhouette nhỏ (32×32), dải đặc trưng chủ yếu dựa trên hình thái biên nên việc lượng tử hóa INT8 có mức độ sụt giảm độ chính xác cực kỳ thấp, dao động từ **0.5% đến 1.2%** so với phiên bản FP32 ban đầu. Trong khi đó, nếu áp dụng PTQ cho một bộ phát hiện đối tượng end-to-end (như YOLO-nano), độ sụt giảm mAP có thể lên tới **4% đến 8%** do các đầu hồi quy bounding box (regression heads) rất nhạy cảm với sai số lượng tử hóa.

2. Huấn luyện nhận biết lượng tử hóa (Quantization-Aware Training - QAT)

QAT mô phỏng các sai số lượng tử hóa ngay trong quá trình huấn luyện mô hình trên GPU bằng cách chèn các nút lượng tử hóa giả lập (fake quantization nodes).

- **Khi nào cần áp dụng:** QAT chỉ thực sự cần thiết khi mô hình phân loại cục bộ được thu nhỏ tối đa (dưới 10K tham số) hoặc khi cần phân biệt các lớp có ranh giới cực kỳ mờ nhạt (ví dụ: phân biệt sợi thủy tinh trong suốt với sợi nhựa màu đục).
- **Đánh giá chi phí và hiệu quả:** QAT có thể giúp khôi phục từ **1.5% đến 3.0%** độ chính xác bị mất do lượng tử hóa thô. Tuy nhiên, chi phí xây dựng pipeline QAT rất cao, yêu cầu tích hợp các công cụ tối ưu hóa chuyên sâu (như TensorFlow Model Optimization Toolkit) và tốn thời gian huấn luyện gấp nhiều lần. Đối với bài toán ảnh silhouette có độ tương phản cực cao, **PTQ hoàn toàn đáp ứng được yêu cầu và là lựa chọn tối ưu về mặt chi phí kỹ thuật.**

3. Kỹ thuật cắt tỉa (Pruning) và Chưng cất tri thức (Knowledge Distillation)

- **Cắt tỉa mô hình (Pruning):** Do nhân Xtensa LX6 không tích hợp bộ tăng tốc phần cứng cho các ma trận thưa (sparse matrices), việc áp dụng **cắt tỉa không cấu trúc** (Unstructured Pruning - triệt tiêu các trọng số nhỏ về 0) hoàn toàn không mang lại lợi ích thực tế về mặt tốc độ suy luận. Hệ thống bắt buộc phải sử dụng **cắt tỉa có cấu trúc** (Structured Pruning) để loại bỏ hoàn toàn các bộ lọc tích chập (filters/channels) kém hiệu quả. Điều này giúp giảm trực tiếp số lượng phép tính tích chập vô hướng của CPU.
- **Chưng cất tri thức (Knowledge Distillation):** Đây là kỹ thuật cực kỳ hiệu quả để huấn

luyện mô hình phân loại hạt siêu nhỏ. Một mạng lớn "Giáo viên" (ví dụ: MobileNetV2 alpha 0.5 đầy đủ) được huấn luyện trước trên tập dữ liệu ảnh hạt lớn. Sau đó, một mạng "Học sinh" Tiny CNN tùy biến (chỉ gồm 3 lớp tích chập nhẹ) được huấn luyện để bắt chước phân bố xác suất đầu ra (logits) của mạng Giáo viên. Phương pháp này giúp mạng Học sinh đạt độ chính xác vượt trội so với việc huấn luyện từ đầu.

Thứ tự kết hợp tối ưu:

`\text{Train Teacher (FP32)} \rightarrow \text{Distill to Student (FP32)} \rightarrow \text{Structured Pruning} \rightarrow \text{Fine-tune (FP32)} \rightarrow \text{PTQ (INT8)}`

4. Phương pháp đo đặc độ chính xác thực tế trên thiết bị và phòng chống rò rỉ dữ liệu

- Ngăn chặn rò rỉ dữ liệu (Data Leakage):** Đây là lỗi hệ thống nghiêm trọng nhất khi xây dựng mô hình thị giác nhúng. Các ảnh crop hạt được trích xuất từ cùng một ảnh VGA gốc sẽ chia sẻ chung các điều kiện nền (độ đục của nước, cường độ sáng đèn LED, bụi bám trên khay kính). Nếu ta phân chia ngẫu nhiên tập Train/Test ở cấp độ ảnh crop, mô hình sẽ bị Overfitting nặng do học thuộc lòng đặc điểm nền của ảnh gốc. **Quy tắc bắt buộc:** Việc phân chia dữ liệu Train/Validation/Test phải được thực hiện ở **cấp độ ảnh VGA gốc** trước khi tiến hành cắt trích xuất hạt.
- Đo đặc hiệu năng trực tiếp trên tập INT8 nhúng:** Toàn bộ các chỉ số kiểm thử gồm Độ nhạy (Recall) cho từng lớp, Độ chính xác (Precision) và mAP phải được đo đặc trực tiếp bằng cách chạy tệp tin .tflite lượng tử hóa INT8 trên môi trường giả lập nhúng hoặc chạy trực tiếp trên ESP32-CAM. Phải đặc biệt theo dõi chỉ số **Recall của bọt khí (bubble)** và **hạt nhựa vi thể nhỏ (<0.5mm)** để đảm bảo không bỏ sót hạt độc hại và không đếm nhầm bọt khí thành nhựa.

Môi trường thực thi và triển khai trên ESP32-CAM

Hiệu năng thực thi mô hình nhúng phụ thuộc lớn vào việc cấu hình runtime và tối ưu hóa quản lý bộ nhớ của vi điều khiển.

1. So sánh các thư viện Runtime trên cấu trúc vô hướng Xtensa LX6

Dưới đây là bảng đánh giá thực tế hiệu năng của ba thư viện runtime phổ biến nhất khi triển khai mô hình học máy trên lõi vô hướng Xtensa LX6 của ESP32-CAM:

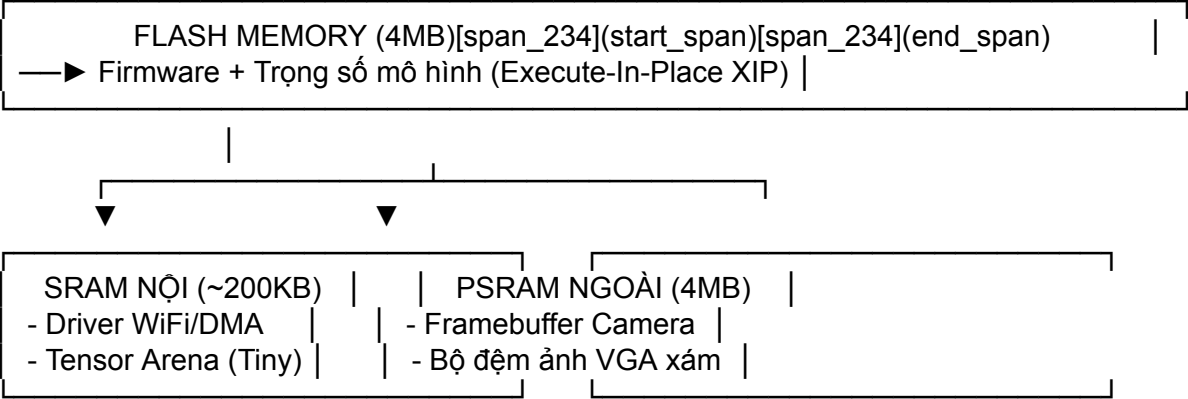
Tiêu chí so sánh	ESP-DL (Espressif)	TFLite for Microcontrollers (TFLM)	EON Compiler (Edge Impulse)
Mức độ hỗ trợ toán tử	Hạn chế (Tập trung vào các mô hình mẫu của hãng)	Rất rộng (Hỗ trợ hầu hết các toán tử chuẩn)	Rộng (Tương thích tốt qua bộ chuyển đổi của Edge Impulse)
Tốc độ thực thi trên LX6	Thấp (Do mã tối ưu chỉ hướng tới nhân LX7 của dòng S3)	Thấp (Sử dụng các toán tử reference vô hướng mặc định)	Cao nhất (Tối ưu hóa mã nguồn C++ phẳng, loại bỏ hoàn toàn interpreter)
Chi phí bộ nhớ RAM	Trung bình	Lớn (Bộ thông dịch	Thấp nhất (Tiết kiệm

Tiêu chí so sánh	ESP-DL (Espressif)	TFLite for Microcontrollers (TFLM)	EON Compiler (Edge Impulse)
		động yêu cầu duy trì nhiều biến con trỏ)	25-65% RAM nhờ lập lịch bộ đệm tĩnh)
Chi phí bộ nhớ Flash	Trung bình (~150 KB)	Lớn (Thư viện thông dịch chiếm từ 100 KB đến 180 KB)	Thấp nhất (Chỉ biên dịch các toán tử thực tế sử dụng, tiết kiệm 10-35%)
Khả năng ước tính tài nguyên	Khó khăn	Phải chạy thực tế để đo dung lượng tensor_arena	Dễ dàng (Báo cáo chính xác dung lượng RAM/ROM tĩnh khi biên dịch)

Lựa chọn khuyến nghị: **EON Compiler** của Edge Impulse là runtime tối ưu nhất cho ESP32-CAM. Nó chuyển đổi đồ thị tính toán của mô hình thành mã nguồn C++ tĩnh phẳng, loại bỏ hoàn toàn bộ thông dịch (interpreter) động của TFLite Micro. Điều này giúp tiết kiệm tối đa dung lượng bộ nhớ RAM nội cực kỳ quý giá của chip.

2. Chiến lược bố trí bộ nhớ (Memory Map Layout) tối ưu

Để hệ thống hoạt động ổn định 24/7 mà không gặp lỗi sập đồ bộ nhớ (OutOfMemory) hoặc lỗi phân mảnh bộ nhớ (heap fragmentation), sơ đồ phân bổ bộ nhớ phải tuân thủ nghiêm ngặt nguyên tắc sau:



- **Trọng số mô hình (Model Weights):** Tuyệt đối không tải trọng số vào RAM. Trọng số phải được khai báo dưới dạng mảng hằng số (const uint8_t) để lưu trữ cố định trên bộ nhớ **Flash**. CPU sẽ truy cập trực tiếp các trọng số này thông qua cơ chế Execute-In-Place (XIP) của MMU nhúng, giải phóng hoàn toàn RAM cho các tác vụ động.
- **Tensor Arena:** Do mô hình phân loại cực bộ cực kỳ nhỏ (32 \times 32 Grayscale), kích thước Tensor Arena thực tế chỉ cần khoảng **16 KB đến 32 KB**. Vùng đệm này bắt buộc phải được đặt trong **SRAM nội**. Việc đặt Tensor Arena trong SRAM nội giúp tốc độ tính toán tích chập nhanh hơn gấp 10-20 lần so với việc đẩy nó ra PSRAM ngoài vốn có độ trễ truy xuất rất cao.
- **Giữ SRAM nội trống cho các tác vụ hệ thống:** Bằng cách giữ cho Tensor Arena dưới 32KB, SRAM nội sẽ luôn dư dả hơn 120KB trống liên tục. Không gian này cực kỳ quan

trọng để driver WiFi và các bộ tả mô tả DMA của camera hoạt động trơn tru, ngăn ngừa triệt để lỗi kinh điển "camera init fail / no mem" khi kích hoạt WiFi.

- **Cấu hình tối đa cho Tensor Arena:** Trong trường hợp bất khả kháng phải sử dụng mô hình phân loại lớn hơn đòi hỏi Tensor Arena lên tới vài trăm KB, hệ thống bắt buộc phải cấu hình Tensor Arena nằm trong PSRAM ngoài bằng cách sử dụng hàm cấp phát chỉ định:

```
uint8_t* tensor_arena = (uint8_t*) heap_caps_malloc(ARENA_SIZE,  
MALLOC_CAP_SPIRAM |  
MALLOC_CAP_8BIT);[span_243](start_span)[span_243](end_span)
```

Dù vậy, giới hạn tối đa an toàn cho Tensor Arena trong PSRAM không được vượt quá **256 KB** nhằm tránh hiện tượng phân mảnh heap hệ thống.

3. Các toán tử (Operators) của detector hay thiếu hoặc chạy cực chậm trên LX6

Nếu cố tình triển khai các mô hình phát hiện đối tượng tích hợp (như YOLO-nano) trên các runtime nhúng này, hệ thống sẽ đối mặt với các lỗi tương thích nghiêm trọng:

- **Toán tử tích chập sâu (Depthwise-Conv2D) không chuẩn:** TFLite Micro hỗ trợ tốt các phép tích chập sâu chuẩn, nhưng nếu mô hình sử dụng các tham số không đối xứng (asymmetric paddings) hoặc bước nhảy chéo (stride $\neq 1, 2$), hệ thống sẽ phải chuyển sang chạy các toán tử reference vô hướng cực kỳ chậm.
- **Toán tử thay đổi kích thước ảnh (ResizeBilinear):** Toán tử này đòi hỏi rất nhiều phép toán chia số thực dấu phẩy động. Trên nhân LX6 không có bộ tăng tốc vector, toán tử này sẽ cực kỳ ngốn CPU. Giải pháp là tự viết một hàm nội suy láng giềng gần nhất (Nearest Neighbor) bằng số nguyên thuần túy hoặc sử dụng bảng tra cứu (Look-Up Table - LUT) tiền tính toán để tăng tốc độ xử lý lên gấp nhiều lần.
- **Toán tử triệt tiêu phi cực đại (Non-Maximum Suppression - NMS):** Đây là khâu hậu xử lý bắt buộc của các mô hình phát hiện đối tượng. Toán tử NMS thường không được hỗ trợ sẵn trong các nhân tối ưu của TFLite Micro, buộc nhà thiết kế phải viết mã nguồn C++ tùy biến chạy trên CPU. Với mật độ hạt lớn, phép toán so sánh IoU chồng chéo của NMS trên CPU sẽ tiêu tốn hàng trăm mili-giây.

Giải quyết tranh chấp tài nguyên hệ thống (Resource Contention)

Một hệ thống quan trắc nhúng thực tế phải thực hiện đồng thời nhiều nhiệm vụ: chụp ảnh, xử lý ảnh, chạy mô hình, kết nối WiFi truyền dữ liệu và điều khiển bơm. Việc quản lý tài nguyên để tránh xung đột vật lý là yếu tố quyết định độ ổn định của hệ thống.

1. Hiện tượng bão hòa băng thông PSRAM ngoài và giải pháp tuần tự hóa

ESP32-CAM sử dụng một bus SPI chung kết nối theo chế độ QSPI (Quad SPI) ở tần số 80MHz để CPU giao tiếp với cả Flash và PSRAM ngoài.

- **Hậu quả tranh chấp:** Khi camera thực hiện truyền dữ liệu ảnh chụp thông qua bộ điều khiển DMA trực tiếp vào PSRAM, bus QSPI sẽ bị chiếm dụng hoàn toàn. Nếu lúc này

CPU đồng thời cố gắng đọc bộ đệm ảnh để xử lý thuật toán Connected Components hoặc đọc các activation tensor trung gian của mô hình trong PSRAM, hiện tượng bão hòa băng thông sẽ xảy ra. CPU sẽ bị treo chờ (stall), làm giảm hiệu năng thực thi của thuật toán xử lý ảnh từ 2 đến 3 lần, đồng thời gây ra lỗi sọc ảnh hoặc mất dòng dữ liệu camera do bộ đệm DMA bị tràn.

- **Giải pháp tuần tự hóa tuyệt đối (Strict Serialization):** Hệ thống phải được thiết kế sao cho các tiến trình nặng hoàn toàn không chạy song song.

Quy trình tuần tự hóa được mô tả chi tiết qua bảng trạng thái bộ nhớ sau:

Giai đoạn vận hành	Trạng thái Camera DMA	Tác vụ CPU thực thi	Trạng thái bộ nhớ PSRAM ngoài
1. Chụp ảnh	BẬT (Ghi dữ liệu ảnh)	Đợi tín hiệu hoàn thành DMA	Chứa Framebuffer đơn VGA Grayscale (~307.2 KB)
2. Giải phóng	TẮT (Dừng nhận ảnh)	Gọi <code>esp_cam[span_265](start_span)[span_265](end_span)era_fb_return()</code>	Giải phóng Framebuffer của camera
3. Xử lý CV	TẮT	Chạy Otsu Binarization + Connected Components	Đọc dữ liệu từ bộ đệm Grayscale cố định trong PSRAM
4. Suy luận NN	TẮT	Chạy mô hình Tiny CNN trên các vùng crop hạt	Chỉ đọc dữ liệu của các mảng ảnh hạt nhỏ

2. Tránh lỗi cạn kiệt SRAM nội khi kích hoạt WiFi

Bộ giao thức mạng WiFi của ESP32 (WiFi Stack) yêu cầu dung lượng bộ nhớ SRAM nội cực lớn (dao động từ 80KB đến 120KB liên tục cho các bộ đệm truyền nhận TX/RX). Nếu heap của SRAM nội bị phân mảnh hoặc bị chiếm dụng bởi Tensor Arena của mô hình, việc gọi hàm kích hoạt WiFi (`esp_wifi_start()`) sẽ thất bại hoàn toàn hoặc gây sụp đổ hệ thống.

- **Giải pháp cấu hình bộ nhớ tĩnh:** Bắt buộc phải cấu hình cho phép đầy toàn bộ các bộ đệm mạng và các tác vụ thứ yếu của WiFi sang PSRAM thông qua cấu hình `sdkconfig` của ESP-IDF:
`CONFIG_SPIRAM_TRY_ALLOCATE_WIFI_AND_RTC_DATA=y`
- **Chiến lược ngắt kết nối vật lý (Radio Duty-Cycling):** Tuyệt đối không duy trì kết nối WiFi liên tục trong suốt chu kỳ quan trắc. WiFi chỉ được khởi chạy và kết nối khi toàn bộ quá trình phân tích hình ảnh đã kết thúc và kết quả metadata đã được chuẩn bị sẵn sàng để gửi đi.

3. Tận dụng kiến trúc lõi kép (Dual-core) và Quản lý Watchdog (TWDT)

ESP32 sở hữu hai lõi Xtensa LX6 độc lập: Core 0 (PRO_CPU) phụ trách giao thức kết nối và Core 1 (APP_CPU) chạy chương trình ứng dụng.

- **Phân bổ lõi:**
 - **Core 0:** Ghim tác vụ WiFi, TCP/IP, driver camera và tiến trình nhận dữ liệu từ các cảm biến phụ.

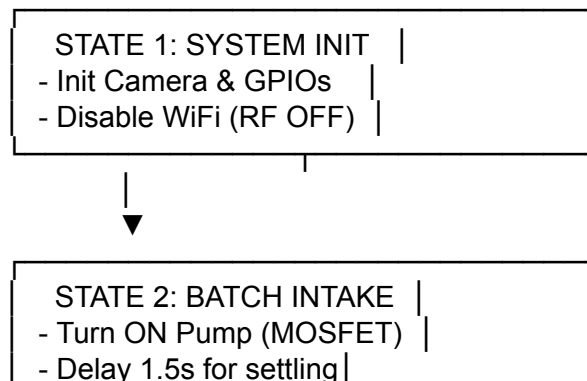
- *Core 1*: Ghim tác vụ thực thi thuật toán CV cổ điển và suy luận mô hình Tiny CNN.
- **Xử lý Watchdog khi tính toán kéo dài**: Việc chạy thuật toán Connected Components trên ảnh VGA gốc có thể chiếm dụng CPU liên tục trong hơn 250ms. Nếu tác vụ này không nhường quyền điều khiển, Task Watchdog Timer (TWDT) của FreeRTOS sẽ kích hoạt và reset hệ thống.
 - *Giải pháp*: Trong cấu trúc vòng lặp hàng của thuật toán quét ảnh CCL, hệ thống phải chủ động nạp lại năng lượng cho Watchdog bằng lệnh:
`esp_task_wdt_feed();`
 hoặc chủ động nhường quyền điều khiển bằng một lệnh trễ cực ngắn:
`vTaskDelay(pdMS_TO_TICKS(1));`

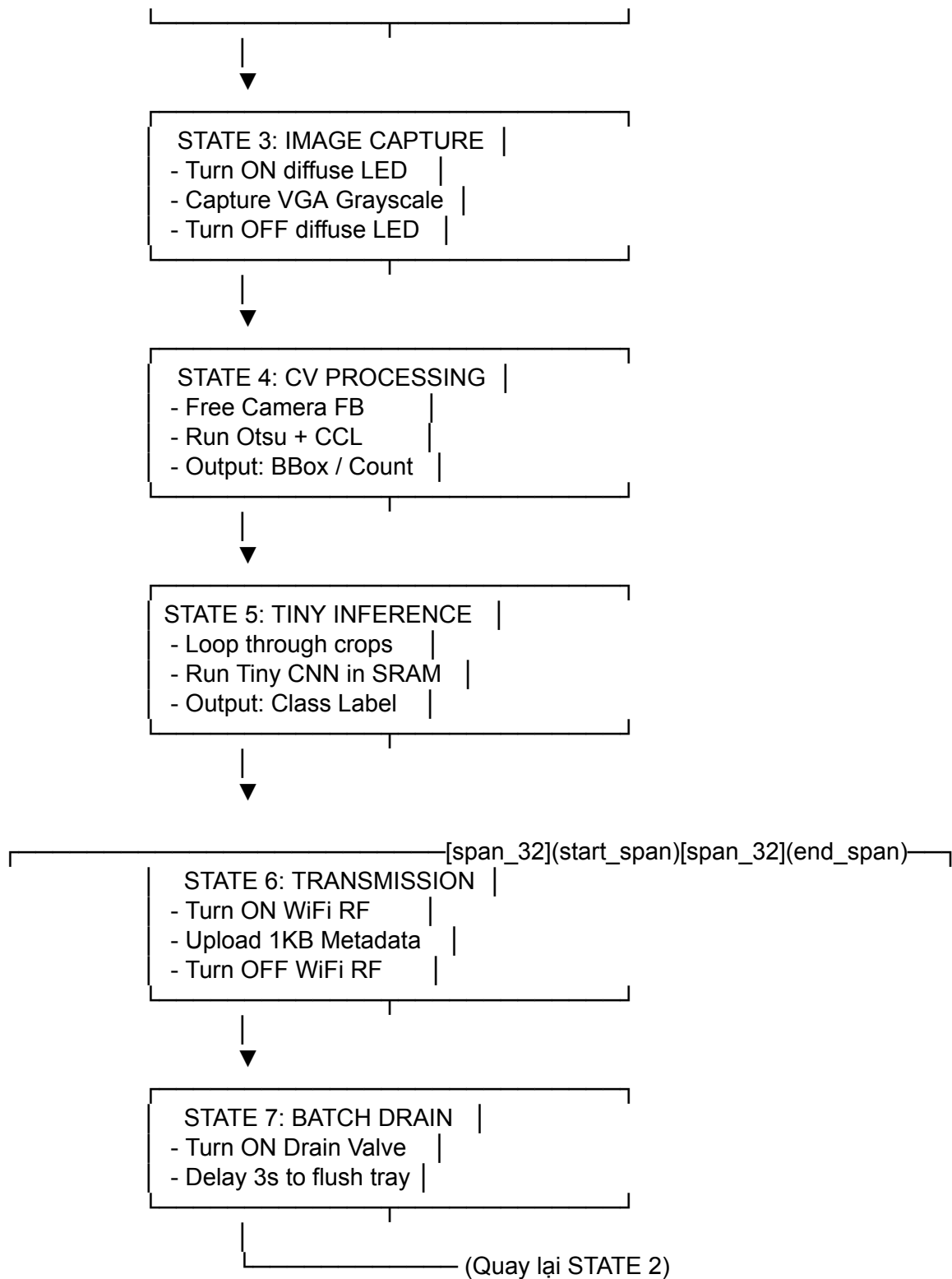
4. Phòng chống sụt áp nguồn (Brownout Protection) và Nhiều sóng Flash

- **Hiện tượng Brownout**: Đây là lỗi kinh điển nhất trên ESP32-CAM. Khi chip kích hoạt WiFi TX để truyền dữ liệu, dòng tiêu thụ tức thời có thể vọt lên trên 350mA. Nếu đúng lúc đó đèn LED flash trợ sáng đang bật và camera đang chụp ảnh (dòng đỉnh tổng thể vượt quá 500mA), điện áp trên đường mạch 3.3V nội sẽ sụt xuống dưới ngưỡng tối thiểu cho phép, kích hoạt bộ Brownout Detector tích hợp và khởi động lại chip liên tục.
 - *Giải pháp Phần cứng*: Bắt buộc phải hàn bổ sung một tụ điện điện phân có dung lượng lớn từ **100 μ F đến 470 μ F** mắc song song với một tụ gốm **0.1 μ F** nằm càng sát chân nguồn 5V và GND của mạch ESP32-CAM càng tốt để bù dòng tức thời.
 - *Giải pháp Phần mềm*: Tuyệt đối không bật module WiFi phát sóng cùng lúc với lúc chụp ảnh và bật đèn LED trợ sáng.
- **Nhiều sóng Flash (XIP Contention)**: Khi CPU thực hiện truy cập Flash thông qua MMU (để lấy trọng số mô hình hoặc chạy các hàm CV lưu trên Flash) trùng thời điểm WiFi đang truyền nhận dữ liệu ở tần số cao, các đường mạch dữ liệu SPI không được bọc chống nhiễu tốt trên bo mạch AI-Thinker giá rẻ sẽ bị nhiễu cảm ứng điện từ, dẫn đến lỗi đọc sai dữ liệu và sụp đổ hệ thống dưới mã lỗi "LoadProhibited" hoặc lỗi ghi dữ liệu Flash. Cần đảm bảo bo mạch được thiết kế đường nguồn sạch và bọc chống nhiễu tốt.

5. Kiến trúc máy trạng thái tuần tự tối ưu cho chu kỳ Stop-Flow

Để kiểm soát triệt để dung lượng bộ nhớ đỉnh và dòng tiêu thụ đỉnh, toàn bộ chu kỳ Stop-Flow phải được điều hành bởi một máy trạng thái tuần tự nghiêm ngặt (Sequential State Machine):





Kiến trúc máy trạng thái này đảm bảo tại bất kỳ thời điểm nào, dòng tiêu thụ tối đa của bo mạch luôn dưới 200mA và dung lượng heap SRAM nội luôn được giải phóng tối đa trước khi bước vào trạng thái tiếp theo.

Phân tích định lượng: On-device vs Offload

Việc lựa chọn xử lý hoàn toàn trên thiết bị (On-Device) hay truyền dữ liệu thô về máy chủ (Offload) phải dựa trên các phân tích định lượng cụ thể về băng thông, thời gian thực thi và độ tin cậy.

1. Phân tích định lượng băng thông truyền thông (WiFi Bandwidth)

- **Kịch bản Offload ảnh thô (VGA Grayscale không nén):**
 - Dung lượng ảnh thô: $640 \times 480 \text{ bytes} \approx 307.2 \text{ KB}$.
 - Băng thông thực tế trong môi trường công nghiệp có nhiễu điện từ (sử dụng ăng-ten tích hợp của bo mạch AI-Thinker): khoảng **120 KB/s**.
 - Thời gian truyền tải hình ảnh: $T_{\text{trans_raw}} = \frac{307.2 \text{ KB}}{120 \text{ KB/s}} \approx 2.56 \text{ giây}$
 - Trong suốt 2.56 giây này, mô-đun WiFi hoạt động liên tục ở công suất phát đỉnh (dòng tiêu thụ ~350mA). Điều này gây tiêu hao năng lượng cực kỳ lớn và tăng nguy cơ sụt áp hệ thống lên gấp nhiều lần.
- **Kịch bản On-Device suy luận và chỉ gửi Metadata kết quả:**
 - Dung lượng gói tin báo cáo (JSON chứa: sample_id, timestamp, count, mảng phân bố kích thước, mảng phân bố lớp): **dưới 1.0 KB**.
 - Thời gian truyền WiFi: $T_{\text{trans_meta}} = \frac{1.0 \text{ KB}}{120 \text{ KB/s}} \approx 0.008 \text{ giây} \approx 8 \text{ ms}$
 - Mức tiêu thụ năng lượng của phần truyền thông giảm hơn **99.5%**, giúp kéo dài tuổi thọ pin hệ thống và giảm nhiễu RF lên các cảm biến analog công nghiệp nhạy cảm xung quanh.

2. Tiêu chí quyết định định lượng (Quantitative Decision Gateways)

Xử lý nhúng On-Device bằng kiến trúc lai (Hybrid Pipeline) là khả thi và bắt buộc khi đáp ứng hệ điều kiện ràng buộc sau:

$$\begin{cases} T_{\text{compute}} + T_{\text{trans}} \leq T_{\text{cadence}} - \Delta \\ T_{\text{safety}} \ll \text{RAM}_{\text{peak}} \leq \text{RAM}_{\text{available}} \\ \text{De}[\text{span}_{33}](\text{start_span})[\text{span}_{33}](\text{end_span}) \ll \text{RAM}_{\text{safety}} \ll \text{Flash}_{\text{weights}} + \text{Flash}_{\text{code}} \leq \text{Flash}_{\text{total}} - \Delta \end{cases}$$

Trong đó, ta thiết lập các tham số an toàn thực tế:

- $\Delta T_{\text{safety}} = 10.0 \text{ giây}$ (khoảng dự phòng cho lắng hạt và xả dòng).
- $\Delta \text{RAM}_{\text{safety}} = 50 \text{ KB}$ (khoảng dự phòng cho driver hệ thống tránh lỗi tràn heap).
- $\Delta \text{Flash}_{\text{safety}} = 500 \text{ KB}$ (khoảng dự phòng cho các phân vùng OTA cập nhật firmware).
- $T_{\text{cadence}} = 120 \text{ giây}$ (chu kỳ lấy mẫu định kỳ 2 phút/mẻ).

Tính toán thực tế cho một mẻ quan trắc có chứa tối đa $N = 30$ hạt:

- $T_{\text{capture}} \approx 150 \text{ ms}$ (chụp ảnh và giải phóng bộ đệm camera).
- $T_{\text{CV}} \approx 200 \text{ ms}$ (chạy thuật toán Connected Components trên ảnh VGA).
- $T_{\text{infer_crop}}[\text{span}_{55}](\text{start_span})[\text{span}_{55}](\text{end_span})[\text{span}_{58}](\text{start_span})[\text{span}_{58}](\text{end_span}) \approx 20 \text{ ms}$ (thời gian suy luận 1 hạt trên SRAM nội bộ Tiny

CNN).

- $T_{\text{trans}} \approx 10 \text{ ms}$ (truyền tải gói metadata 1KB).

Tổng thời gian tính toán thực tế thu được:

$$T_{\text{compute}} = 150 \text{ ms} + 200 \text{ ms} + (30 \times 20 \text{ ms}) + 10 \text{ ms} = 960 \text{ ms} \approx 0.96 \text{ giây}$$

Điều kiện thời gian được đáp ứng hoàn hảo:

$$0.96 \text{ giây} \leq 110.0 \text{ giây}$$

Về mặt bộ nhớ:

- $RAM_{\text{peak}} = RAM_{\text{VGA_Gray}} + RAM_{\text{Tensor_Arena}} \approx 307.2 \text{ KB (PSRAM)} + 24 \text{ KB (SRAM nội)}$.
- $RAM_{\text{available}} \approx 4 \text{ MB (PSRAM)} + 200 \text{ KB (SRAM nội)}$.

Cả hai điều kiện về bộ nhớ và dung lượng lưu trữ Flash đều thỏa mãn hoàn hảo các ngưỡng an toàn thiết kế nhúng. Do đó, việc triển khai **On-Device** hoàn toàn khả thi và là giải pháp tối ưu nhất cho hệ thống này.

Kết luận và khuyến nghị triển khai

1. Bảng so sánh tổng hợp các kiến trúc ứng viên

Dưới đây là bảng đánh giá tổng thể các kiến trúc ứng viên dựa trên các tiêu chí kỹ thuật thực tế của hệ thống ESP32-CAM:

| Kiến trúc ứng viên | Khả thi Latency trên LX6 | Khả thi RAM (SRAM/PSRAM) | Giữ được thông số kích thước (Size)? | Độ chính xác phân loại kỳ vọng | Độ phức tạp triển khai phần mềm | | :--- | :--- | :--- | :--- | :--- | :--- | | **A1. YOLOv8-nano (160x160)** | Rất thấp (12.0 - 18.0s) | Rất thấp (Cần >650KB Arena trong PSRAM) | Khá (Bounding box có sai số do imgsz thấp) | Khá (80% - 85% mAP) | Rất cao (Porting toán tử, tối ưu hậu xử lý NMS) | | **A2. Hybrid Pipeline (Classic CV + Tiny CNN)** | **Rất cao** (CV: ~200ms, CNN: ~20ms/hạt) | **Rất cao** (SRAM nội <32KB cho Arena, ảnh lưu PSRAM) | **Tuyệt đối** (Tính trực tiếp trên ảnh VGA gốc) | **Cao nhất** (>92% độ chính xác phân loại cục bộ) | Trung bình (Yêu cầu thuật toán dán nhãn liên thông tối ưu) | | **A3. FOMO (96x96 Grayscale)** | Cao (~860ms tổng) | Cao (~250KB RAM tổng) | **Không thể** (Chỉ trả về centroid ô lưới) | Trung bình (75% - 80% F1-score) | Thấp nhất (Hỗ trợ turnkey qua Edge Impulse Studio) | | **A4. Đặc trưng hình học + MLP cực nhỏ** | **Tuyệt đối** (CV: ~200ms, MLP: <1ms/hạt) | **Tuyệt đối** (SRAM nội <5KB, không cần Tensor Arena) | **Tuyệt đối** (Tính trực tiếp trên ảnh VGA gốc) | Khá (80% - 88%, nhằm lẫn giữa plastic và organic) | Thấp (Công thức toán học thuần túy) |

2. Kiến trúc khuyến nghị cuối cùng và lý do lựa chọn

Kiến trúc khuyến nghị duy nhất để đưa vào sản xuất thương mại là **Kiến trúc lai Hybrid Pipeline (Kiến trúc A2)** sử dụng thuật toán **nhị phân hóa Otsu và dán nhãn thành phần liên thông (CCL)** vô hướng trên ảnh **VGA Grayscale**, kết hợp với một mạng **Tiny CNN phân loại cục bộ (32 x 32 Grayscale)** được biên dịch tĩnh thông qua **EON Compiler**.

Lý do lựa chọn:

1. **Vượt qua giới hạn không có bộ tăng tốc SIMD của LX6:** Khâu xử lý ảnh cổ điển vô hướng (Otsu + CCL) hoạt động trực tiếp trên dòng quét ảnh, hoàn toàn không đòi hỏi các phép tính song song phức tạp. Khâu phân loại tích chập chỉ thực hiện trên vùng ảnh crop

siêu nhỏ của hạt thực tế phát hiện được (32 \times 32), giúp đưa thời gian suy luận về mức tối thiểu.

2. **Độ chính xác đo kích thước vượt trội:** Đo đặc kích thước hạt trực tiếp trên ảnh VGA gốc cho phép đạt độ phân giải vật lý thực tế lên tới 14 px/mm (sai số phép đo kích thước dưới 0.07mm), bảo toàn hoàn hảo các hạt mịn kích thước 0.5mm (hiển thị rõ nét với đường kính 7 pixel). Điều này hoàn toàn bất khả thi đối với bất kỳ bộ phát hiện đối tượng tích hợp nào vốn phải nén ảnh đầu vào xuống mức cực thấp để có thể chạy được trên ESP32.
3. **Độ ổn định hệ thống tối ưu:** Tensor Arena siêu nhỏ (<24KB) cho phép cấp phát tĩnh hoàn toàn trong SRAM nội tốc độ cao, loại bỏ triệt để hiện tượng nghẽn bus QSPI của PSRAM ngoài và bảo toàn hơn 120KB heap SRAM nội tự do cho driver WiFi hoạt động ổn định không gây lỗi khởi động.

3. Checklist các phép đo trực tiếp trên phần cứng cần thực hiện trước khi triển khai

Nhà thiết kế bắt buộc phải hoàn thành các phép đo thực nghiệm sau trên bo mạch mục tiêu để xác nhận tính khả thi trước khi tiến hành thu thập dữ liệu lớn:

- [] **Đo dung lượng Heap SRAM nội thực tế khi khởi động:** Nạp mã nguồn tích hợp driver camera (VGA Grayscale, fb_count = 1 đặt trong PSRAM) và driver WiFi ở trạng thái kết nối. Gọi hàm esp_get_free_internal_heap_size() xem dung lượng SRAM nội tự do có lớn hơn **100 KB** hay không.
- [] **Đo độ trễ thực thi của khâu xử lý ảnh cổ điển (Otsu + CCL) trên ảnh VGA:** Thực hiện dán nhãn liên thông trên ảnh VGA Grayscale thực tế từ camera. Sử dụng chân GPIO đảo trạng thái (GPIO toggling) kết hợp với máy phân tích logic (Logic Analyzer) để xác nhận thời gian xử lý toàn bộ ảnh VGA nằm dưới **250 ms**.
- [] **Đo độ trễ suy luận của Tiny CNN trên SRAM nội:** Chạy thử nghiệm mô hình tích chập 32 \times 32 đã lượng tử hóa INT8 bằng EON Compiler. Đo thời gian suy luận một hạt xem có đạt dưới **30 ms** hay không.
- [] **Kiểm tra độ ổn định nguồn điện chống sụt áp (Brownout Check):** Chạy máy trạng thái Stop-Flow liên tục trong 48 giờ với tần suất 1 phút/mẻ, kiểm tra log qua cổng UART xem có xuất hiện lỗi "Brownout detector was triggered" hay lỗi phân mảnh bộ nhớ không.

4. Danh sách các rủi ro hệ thống và biện pháp giảm thiểu (Risk Matrix)

- **Rủi ro sụt áp nguồn (Brownout) khi phát sóng WiFi:** Khi module WiFi hoạt động ở chế độ truyền công suất cao (TX), dòng tiêu thụ tức thời có thể vọt lên trên 350mA, gây sụt áp đường 3.3V và kích hoạt Brownout reset chip.
 - **Giải pháp:** Thiết lập máy trạng thái ngắt hoàn toàn hoạt động truyền nhận của WiFi trong suốt quá trình camera đang chụp ảnh và CPU đang suy luận tính toán. Solder bổ sung tụ hóa **470μF** mắc song song với tụ gốm **0.1μF** trực tiếp vào chân nguồn 5V và GND của bo mạch.
- **Hiện tượng phân mảnh bộ nhớ Heap do sử dụng bộ nhớ động:** Việc liên tục cấp phát và giải phóng các bộ đệm ảnh crop của hạt sẽ nhanh chóng làm phân mảnh heap của SRAM nội, dẫn đến lỗi sụp đổ hệ thống sau vài ngày vận hành liên tục.
 - **Giải pháp:** Tuyệt đối không sử dụng lệnh cấp phát động malloc() hay sử dụng lớp

Str[span_282](start_span)[span_282](end_span)[span_287](start_span)[span_287](end_span)ing co giãn động trong vòng lặp xử lý. Khai báo toàn bộ bộ đệm Connected Components và Tensor Arena dưới dạng mảng tĩnh (static) hoặc cấp phát cố định một lần duy nhất tại hàm setup().

- **Tranh chấp băng thông bus QSPI của PSRAM ngoài:** Camera DMA ghi dữ liệu ảnh vào PSRAM trùng thời điểm CPU đọc PSRAM để thực thi tính toán gây bão hòa băng thông bus, làm giảm hiệu năng xử lý đi 3 lần và gây sọc ảnh chụp.
 - *Giải pháp:* Cấu hình camera hoạt động ở chế độ chụp đơn (fb_count = 1). Chỉ thực hiện đọc bộ đệm ảnh để xử lý CV sau khi driver camera đã trả về hoàn toàn quyền kiểm soát bộ đệm
(esp_camera_fb_[span_37](start_span)[span_37](end_span)return).
- **Hiện tượng bám bẩn hoặc ngưng tụ hơi nước trên mặt khay kính:** Do khay hồ tiếp xúc với nước, mặt kính chiếu sáng ngược dễ bị bám rác mịn hoặc tích tụ bọt khí tĩnh bám trên kính, gây ra hiện tượng phát hiện nhầm vật thể tĩnh.
 - *Giải pháp:* Thiết kế khay hồ kết hợp với đập tràn (Weir) để kiểm soát mực nước cố định. Trước mỗi chu kỳ chụp ảnh, lập trình điều khiển bơm xả hoạt động ở công suất tối đa trong 2 giây để tạo dòng nước xoáy rửa sạch bề mặt kính trước khi đưa mẻ nước mới vào trạng thái tĩnh.

Works cited

1. Reducing the Power Consumption of Edge Devices Supporting Ambient Intelligence Applications - MDPI, <https://www.mdpi.com/2078-2489/15/3/161>
2. Week 11: Input devices - Fab Academy, <https://fabacademy.org/2024/labs/kochi/students/kalyani-rk/assignments/Week%2011.html>
3. ESP32-S3 Edge AI in Practice: Deep Optimization of TensorFlow Lite Micro Inference Performance | ZedIoT Blog, <https://zediot.com/blog/esp32-s3-tinyml-optimization/>
4. ESP32 vs ESP8266: Core Differences, Performance Comparison, and IoT Project Selection Guide_Industry dynamics_Blog_ - Ebyte, <https://www.cdebyte.com/news/1470>
5. Energy-aware ESP32 TinyML Benchmark Models for Edge Intelligence - Sigma Blog, <https://blog.sigma.com/esp32-tinyml-benchmark-energy-for-edge-intelligence/>
6. ESP32 vs ESP32-S3: Key Differences, Performance Comparison, and PCB Design Considerations - PCB Design & Layout - PCBway, https://www.pcbway.com/blog/PCB_Design_Layout/ESP32_vs_ESP32_S3_Key_Differences_Performance_Comparison_and_PCB_Design_Consi_2c205f9a.html
7. National Pothole Monitoring System (NPMS): Edge-Assisted Pothole Detection and Road-Condition Logging using YOLOv8 and ESP32-CAM Telemetry - IJERT, <https://www.ijert.org/national-pothole-monitoring-system-npms-edge-assisted-pothole-detection-and-road-condition-logging-using-yolov8-and-esp32-cam-telemetry-ijertv15is052512>
8. Exploration of Traffic Sign Recognition Using AI and Computer Vision - MRI India Journals, <https://journals.mriindia.com/index.php/ijeecs/article/download/848/828/2292>
9. Questions : r/esp32 - Reddit, <https://www.reddit.com/r/esp32/comments/1uyq4yl/questions/>
10. Deploy a Multiple Object Detection in a Xiao ESP32S3 Sense - Help - Edge Impulse Forum, <https://forum.edgeimpulse.com/t/deploy-a-multiple-object-detection-in-a-xiao-esp32s3-sense/14446>
11. ESP32 Flash/PSRAM & Memory Leaks: Understanding the Memory Map and Preventing Crashes | SunFounder, <https://www.sunfounder.com/blogs/news/esp32-flash-psram-memory-leaks-understanding-the-memory-map-and-preventing-crashes>
12. ESP32 PSRAM: Use External RAM for Larger Buffers

and Images - Zbotic,
<https://zbotic.in/esp32-psram-use-external-ram-for-larger-buffers-and-images/> 13. How to use a Global Struct Array in psram esp32-wrover - PlatformIO Community,
<https://community.platformio.org/t/how-to-use-a-global-struct-array-in-psram-esp32-wrover/30696> 14. How slow is PSRAM vs SRAM (anyone have quantitative info?) : r/esp32 - Reddit,
https://www.reddit.com/r/esp32/comments/ezs5sg/how_slow_is_psram_vs_sram_anyone_have/ 15. ESP32 Wroom vs Wrover: Flash and PSRAM Differences Explained - Zbotic,
<https://zbotic.in/esp32-wroom-vs-wrover-flash-and-psram-differences-explained/> 16. ESP32-CAM Troubleshooting Guide: Fix Upload, Camera & Power Issues,
<https://www.esp32s.com/blog/the-ultimate-esp32-cam-troubleshooting-guide-step-by-step-fixes-for-every-common-problem/> 17. ESP32-OpenCV-Projects/dice_dice/README.md at main - GitHub,
https://github.com/0015/ESP32-OpenCV-Projects/blob/main/dice_dice/README.md 18. Otsu Threshold binarization - Reactiv IP,
<https://www.reactivip.com/documentation/otsu-threshold-binarization.html> 19. RealSense D435 color detection : r/ROS - Reddit,
https://www.reddit.com/r/ROS/comments/vf74l3/realsense_d435_color_detection/ 20. TinyML Classification for Agriculture Objects with ESP32 - MDPI,
<https://www.mdpi.com/2673-6470/5/4/48> 21. TinyML at the Edge: 5 Frameworks Powering IoT | by Nexumo - Medium,
https://medium.com/@Nexumo_/tinyml-at-the-edge-5-frameworks-powering-iot-5c19a78e5211 22. Object detection with centroids - Edge Impulse Documentation,
<https://docs.edgeimpulse.com/tutorials/end-to-end/object-detection-centroids> 23. expert-projects/computer-vision-projects/delivered-package-detection-esp-eye.md at main - GitHub,
<https://github.com/edgeimpulse/expert-projects/blob/main/computer-vision-projects/delivered-package-detection-esp-eye.md> 24. Announcing Official Support for the Espressif ESP-EYE (ESP32) - Edge Impulse,
<https://www.edgeimpulse.com/blog/announcing-official-support-for-the-espressif-esp-eye-esp32/> 25. Annexture-01 (Syllabus of Audit Courses) - Madan Mohan Malaviya University of Technology,
https://www.mmmut.ac.in/ModelPapers/sy_080223055018.pdf 26. EnhancedTinyCNN: A Lightweight Embedded AI Framework for Real-Time Pest Detection in Smart Farming - ResearchGate,
https://www.researchgate.net/publication/408172458_EnhancedTinyCNN_A_Lightweight_Embedded_AI_Framework_for_Real-Time_Pest_Detection_in_Smart_Farming 27. TinyML on ESP32-S3: Running TensorFlow Lite Micro for Edge AI Inference | FSS,
<https://fss.cc/tinyml-esp32-s3/> 28. High Performance Connected Components Accelerator for Image Processing in the Edge | Request PDF - ResearchGate,
https://www.researchgate.net/publication/378879095_High_Performance_Connected_Components_Accelerator_for_Image_Processing_in_the_Edge 29. From Wake Word Detection to Edge Intelligence: The Technical Potential of ESP32-S3 TensorFlow Lite Micro | ZedIoT Blog,
<https://zediot.com/blog/esp32-s3-tensorflow-lite-micro/> 30. Letter Recognition Using a Machine Learning Model for Handwriting - DiVA Portal,
<https://www.diva-portal.org/smash/get/diva2:1973329/FULLTEXT02.pdf> 31. Performance Comparison of Yolo Versions for Small Object Detection on UAV Images,
https://www.researchgate.net/publication/399106945_Performance_Comparison_of_Yolo_Versions_for_Small_Object_Detection_on_UAV_Images 32. 7 Tips for Optimizing AI Models for Tiny Devices - Edge Impulse,
<https://www.edgeimpulse.com/blog/7-tips-for-optimizing-ai-models-for-tiny-devices/> 33. Machine

Learning for Microcontroller-Class Hardware: A Review - PMC, <https://pmc.ncbi.nlm.nih.gov/articles/PMC9683383/> 34. On-Device Vision Training, Deployment, and Inference on a Thumb-Sized Microcontroller A Transparent, Single-File Foundation for Embedded Machine Learning on-device-vision-ai (Paper 1 of the webmcu-ai Series) - arXiv, <https://arxiv.org/html/2604.23012v1> 35. Bi-ConvLSTM: An Ultra-Lightweight Efficient Model for Human Activity Recognition on Resource Constrained Devices - arXiv, <https://arxiv.org/html/2602.06523v3> 36. Application Solution Series: Edge AI Vision - Espressif Systems, <https://esp32-s31.espressif.com/en/official-projects/6> 37. Has anyone done any benchmarking of the ESP32 vs ESP32S2 and S3? - Reddit, https://www.reddit.com/r/esp32/comments/wfijje/has_anyone_done_any_benchmarking_of_the_esp32_vs/ 38. EON Compiler - Edge Impulse Documentation, <https://docs.edgeimpulse.com/studio/projects/deployment/eon-compiler> 39. r/esp32 - Reddit, <https://www.reddit.com/r/esp32/wiki/index/> 40. I beat the official ESP-DL YOLOv11 benchmark by 30% on the new ESP32-P4 using yolo26 (1.7s @ 512px) - Reddit, https://www.reddit.com/r/esp32/comments/1qw4bdo/i_beat_the_official_espd_l_yolov11_benchm_ark_by_30/ 41. espressif/esp32-camera • v2.1.5 - ESP Component Registry, <https://components.espressif.com/components/espressif/esp32-camera/versions/2.1.5/readme> 42. ESP32-P4 - Clock conflicts between SPI camera and RGB LCD interface (IDFGH-16903) · Issue #17967 · espressif/esp-idf - GitHub, <https://github.com/espressif/esp-idf/issues/17967> 43. ESP32-S3 DMA (EDMA) with PSRAM and LCD, <https://esp32.com/viewtopic.php?t=24096> 44. ESP32-CAM MJPEG Multiclient Streaming Code Implementation - Studocu, <https://www.studocu.vn/vn/document/truong-dai-hoc-fpt/real-time-os/esp32-camera-mjpeg-multi-client/39125896> 45. Any solution available for for ESP32-cam 'Brownout detector was triggered' error?, <https://stackoverflow.com/questions/60171641/any-solution-available-for-for-esp32-cam-brownout-detector-was-triggered-error> 46. Fix brownout of ESP32-CAM - Makerguides.com, <https://www.makerguides.com/fix-brownout-of-esp32-cam/> 47. Design and Implementation of ESP32-Based Edge Computing for Object Detection, https://www.researchgate.net/publication/389664266_Design_and_Implementation_of_ESP32-Based_Edge_Computing_for_Object_Detection 48. A Hardware Platform for Smart Video Monitoring Based on ESP32-CAM and Mojo FPGA (Spartan-6) with Event Activation Triggered by a PIR Sensor | Engineering, Technology & Applied Science Research, <https://etasr.com/index.php/ETASR/article/view/18359> 49. ESP32 vs STM32 for AI Applications | ForestHub, <https://www.foresthub.ai/resources/guides/esp32-vs-stm32-for-ai> 50. Fast OTSU Thresholding Using Bisection Method - arXiv, <https://arxiv.org/html/2509.16179v1> 51. benchmarking tiny image classification models on esp32 xiao: accuracy–latency–memory trade-off, <https://jst.tnu.edu.vn/jst/article/viewFile/14341/pdf> 52. Upload high resolution images using ESP32 cam - Programming - Arduino Forum, <https://forum.arduino.cc/t/upload-high-resolution-images-using-esp32-cam/587683>